

7-MDP 策略迭代

马尔可夫决策过程 MDP

解决MDP的两种算法：

- 价值迭代算法 [6-MDP 价值迭代](#)
- 策略迭代算法

策略评估和策略提取

为了评估不同策略的优劣，定义一个固定的策略下的效用值：

$$V^{\pi}(s) = \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V^{\pi}(s')]$$

遵守一个特定的策略 π 时， V 是相似的，只不过没有了 $\max_a$ ，因为 a 由 $\pi(s)$ 决定，这实际上已经是一个线性方程

策略评估 Policy Evaluation

有了上面固定策略下的效用值迭代式子，就可以通过类似价值迭代的方法得到某个 V^{π}

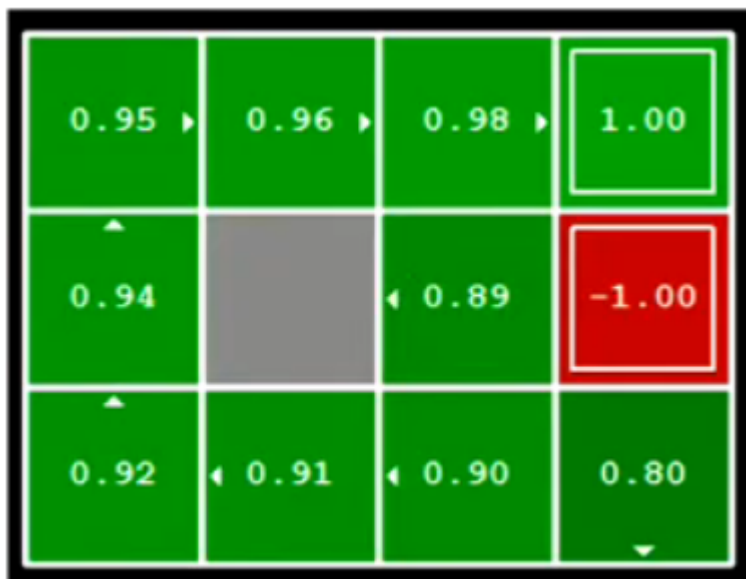
$$\text{令 } V_0^{\pi}(s) = 0$$

$$V_{k+1}^{\pi}(s) = \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V_k^{\pi}(s')]$$

因为少了对所有action遍历求最大，复杂度是 $O(s^2)$

策略提取 Policy Extraction

如果已经知道各个状态下的 V^* ，该怎么得到最优的策略？



这并不是显然的向着value大的地方移动就行，而要遍历所有可能的action，得到哪个action能得到最大的 V ：

$$\pi^*(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

(argmax 返回取到最大值的参数a)

只不过因为各个状态下的V*已知，公式里的V*(s')是已知的，相当于一步expectimax
特别的，如果知道是Q*，那么可以直接知道策略，因为qstate就是action后的价值

策略迭代算法 Policy Iteration

价值迭代存在以下问题（建议看BV1HcqpYwEw6 55min处的讲解）：

- $O(S^2 A)$ 的复杂度太高，尤其是State多的时候
- 往往在价值收敛之前，策略就已经得到了收敛

所以给出策略迭代算法，该算法重复策略评估和策略提取两步，直到策略收敛：

$$V_{k+1}^{\pi_i}(s) = \sum_{s'} T(s, \pi_i(s), s') [R(s, \pi_i(s), s') + \gamma V_k^{\pi_i}(s')]$$

上式进行到V收敛为止

$$\pi_{i+1}^*(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k^{\pi_i}(s')]$$

上式根据第一步给出的value提取出最优的策略

- 最优的
- 特定条件下收敛快很多

总结

上面提到的内容其实都是贝尔曼方程的变体，也都是一层的expectimax（迭代，V_k已经得知）[5-对抗搜索 Expectimax](#)，区别只是遍历了action还是有个确定的策略